



UNIVERSITY OF BUEA
FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING
CEF440 INTERNET PROGRAMMING AND MOBILE PROGRAMMING

TASK 7:FINAL PRESENTATION

Group 15

COURSE INSTRUCTOR: Dr Nkemeni Valery

NAME	MATRICULE
NJINYA NGECHA RYAN BROWN	FE23A116
KARL JONAS WENGE ACHA	FE23A072
TAKU-MBI INYANG	FE23A153
BRIANNA TEBESICK TSAMO	FE23A028
KELSEY NJOCK-AWOH NJOCK OBEN	FE23A073

Table of content

Table of Contents.....	Error! Bookmark not defined.
1. Introduction	4
2. Task 1: Foundations of Mobile Application Development	4
2.1 Types of Mobile Applications	4
2.2 Programming Languages and Frameworks.....	5
2.3 Mobile Application Architectures and Design Patterns	5
2.4 Requirement Engineering for Mobile Applications	5
2.5 Estimating Mobile Application Development Cost	6
3. Task 2: Requirements Gathering	6
3.1 Stakeholder Identification	6
Primary Stakeholders	6
Secondary Stakeholders	6
3.2 Requirements Gathering Techniques.....	6
Key Findings from Reverse Engineering	7
3.3 Data Gathering and Cleaning	7
3.4 User Reluctance Assessment.....	7
4. Task 3: Requirements Analysis and Software Requirements Specification	8
4.1 Analysis of Survey Findings	8
4.2 Review and Analysis of Requirements.....	8
4.3 Prioritisation of Requirements.....	8
4.4 Classification of Requirements.....	9
Functional Requirements	9
Non-Functional Requirements	10
4.5 Software Requirements Specification (SRS) Summary	10
Constraints	10
Key Assumptions.....	10
5. Task 4: System Modelling and Design.....	11
5.1 Use Case Diagram	11
5.2 Class Diagram.....	11
5.3 Deployment Diagram.....	12
5.4 Sequence Diagrams	12
5.5 Data Flow Diagram (Level 1)	12
5.6 Context Diagram	12
6. Task 5: User Interface Design and Frontend Implementation	13

6.1 Brand Identity	13
6.2 Visual Design	13
6.3 UI Components	13
6.4 Frontend Technology Stack	14
6.5 Role-Based Dashboards	14
6.6 Outcome.....	16
7. Task 6: Database Design and Backend Implementation	16
7.1 Data Elements	17
7.2 Conceptual Design	17
7.3 Entity-Relationship Diagram	18
7.4 Database Implementation.....	19
7.5 Backend Implementation	19
7.6 Connecting the Database to the Backend.....	20
7.7 Testing	20
7.8 Challenges Encountered	20
7.9 Outcome.....	20
8. Overall Conclusion and Recommendations	21
8.1 Current State of the Platform.....	21
8.2 Recommendations for Further Work	Error! Bookmark not defined.

1. Introduction

This report brings together all the work carried out by Group 15 during the CEF440 (Internet Programming and Mobile Programming) course, from the very first task to the final backend implementation task. Over the course of the semester, the group progressively researched, designed, and built LearnBridge, an adaptive e-learning platform based on Quality of Experience (QoE) for inclusive online education in low-resource settings.

The project was carried out in six stages (Tasks 1 to 7), each building on the results of the previous one:

- Task 1 — Foundational research into mobile application types, programming languages, frameworks, architectures, design patterns, requirement engineering, and cost estimation for mobile applications.
- Task 2 — Identification of stakeholders and collection of requirements data through brainstorming, observation, reverse engineering of existing platforms, and surveys.
- Task 3 — Analysis of the data gathered in Task 2, classification and prioritisation of requirements, and the development of a full Software Requirements Specification (SRS).
- Task 4 — System modelling and design, including use case diagrams, class diagrams, deployment diagrams, sequence diagrams, a data flow diagram, and a context diagram.
- Task 5 — User interface design and frontend implementation of the LearnBridge platform using React, Next.js, TypeScript, Tailwind CSS, and Zustand.
- Task 6 — Database design and backend implementation using PostgreSQL, Prisma ORM, Node.js, and Express.js, including authentication and route testing.
- Task 7—Final presentation

The remaining sections of this report present each task in turn, summarising the objectives, methodology, key findings, and deliverables produced, before closing with an overall conclusion on the state of the LearnBridge platform and recommendations for further work.

2. Task 1: Foundations of Mobile Application Development

Before any design or development work began, the we carried out background research into the mobile application landscape. This foundational task ensured that all subsequent design decisions (Tasks 4-6) were grounded in a solid understanding of the available technologies and engineering practices.

2.1 Types of Mobile Applications

Mobile applications were classified according to how they are built and delivered: native apps, progressive web apps (PWAs), and hybrid apps. Each category was compared in terms of architecture, performance, cost, development time, access to device features, and distribution.

- Native apps are built for a single operating system using platform-specific languages (Java/Kotlin for Android, Swift for iOS). They offer the best performance, full device access and strong security, but are expensive to build and maintain because separate codebases are required per platform.
- Progressive Web Apps (PWAs) are advanced, browser-based applications that can behave like native apps — they can be “installed”, work offline via service workers, and are delivered through a single codebase without requiring an app-store download.
- Hybrid apps wrap web technologies inside a native shell (e.g. using frameworks such as Ionic or Cordova), offering a middle ground between native performance and cross-platform reuse of code.

This comparison directly informed the technology constraint adopted later in Task 3 and Task 4: because LearnBridge needed to reach low-end Android smartphones with limited storage and unreliable connectivity, a Progressive Web App architecture was identified early on as the most suitable direction.

2.2 Programming Languages and Frameworks

The group reviewed the major languages used in mobile development — Java and Kotlin for Android, Swift for iOS, Dart (Flutter) and JavaScript (React Native) for cross-platform development, C# for .NET MAUI/Xamarin, along with Python and C++ — comparing them on performance, learning curve, ecosystem maturity, and suitability for cross-platform delivery. Mobile frameworks were similarly reviewed and compared, leading to use-case recommendations for when a native, cross-platform, or web-based framework would be the better choice.

2.3 Mobile Application Architectures and Design Patterns

The group studied the architectural patterns commonly used in mobile and web application development, including MVC (Model-View-Controller), MVP (Model-View-Presenter), MVVM (Model-View-ViewModel), MVI (Model-View-Intent), Clean Architecture, and VIPER. Alongside these, core design patterns relevant to scalable application development were reviewed: Singleton, Observer, Repository, and Dependency Injection. This groundwork later shaped the layered, role-based structure adopted in the LearnBridge backend (controllers, routes, middleware) in Task 6.

2.4 Requirement Engineering for Mobile Applications

The group studied the requirement engineering lifecycle for mobile systems — elicitation, analysis and classification, documentation and specification, and validation — as well as the challenges unique to mobile requirement engineering, such as fragmented device capabilities, variable network conditions, and diverse user contexts. This theoretical grounding was applied directly in Tasks 2 and 3 of the project.

2.5 Estimating Mobile Application Development Cost

Finally, the group reviewed the primary cost drivers for mobile app development (team size, platform complexity, feature scope, integration requirements) and common estimation techniques and formulas used to project development effort and cost.

3. Task 2: Requirements Gathering

Building on the research from Task 1, the group turned its attention to the specific problem the project would address: poor network quality and unstable power supply as barriers to e-learning in many African countries. The goal of Task 2 was to design an adaptive mobile e-learning platform that dynamically adjusts content delivery and communication features based on network conditions, in order to optimise Quality of Experience (QoE) in low-resource environments.

3.1 Stakeholder Identification

The group identified five stakeholder groups, divided into primary and secondary stakeholders.

Primary Stakeholders

- Diverse Learners — students with varying levels of digital literacy, socio-economic backgrounds, and abilities, across all educational levels.
- Instructors (Educators and Facilitators) — university lecturers and school teachers who create and manage content.
- Parents & Guardians — who are concerned with their children's academic progress and who fund their education, particularly in low-resource settings.
- School Administrators — proprietors, head teachers, principals, heads of department, deans, and vice-chancellors.

Secondary Stakeholders

- Ministries in charge of Education — in Cameroon, this includes the ministries of basic, secondary, and higher education, which regulate and support the educational system through policy.

3.2 Requirements Gathering Techniques

Four complementary techniques were used to gather requirements:

- Brainstorming — a team session to identify possible features and challenges, surfacing issues such as poor network quality, limited power supply, limited device availability, and the need for offline learning support.
- Observation — direct observation of how lecturers teach, the tools they use, lesson duration, and their comfort with technology, as well as how students interact with lecturers and peers.

- Reverse Engineering — a structured study of four existing e-learning systems (including Eneza and Moodle) to understand their features, their performance under low-network conditions, and their limitations.
- Surveys — the primary data collection method, used to gather structured responses from learners, instructors, parents, and school administrators.

Key Findings from Reverse Engineering

Eneza was found to be designed specifically for low-resource environments, delivering lessons and quizzes via SMS, USSD, and mobile web, with an “Ask a Teacher” feature and performance dashboards. However, it is limited mostly to text-based content, depends on airtime/network availability, and does not address device or electricity shortages. Moodle, in contrast, offers rich course management, discussion forums, a mobile app, and offline access to materials, but is heavier and less adapted to extremely constrained network conditions. These findings directly shaped the requirement for an adaptive, multimedia-capable, offline-first platform in later tasks.

3.3 Data Gathering and Cleaning

The survey collected 95 responses in total: 82 from learners, 6 from instructors, 4 from parents, and 3 from school administrators. Alongside the survey, the team studied four existing systems (Eneza, Moodle, Kolibri, and YouTube) as reference points. After collection, the data was cleaned to remove incomplete responses, duplicate entries, and invalid data, leaving only valid responses for analysis.

3.4 User Reluctance Assessment

The team also assessed the reasons users might be reluctant to adopt an online learning platform. During data collection itself, some respondents were hesitant to click survey links for fear their devices could be compromised, while some teachers and parents felt the survey did not apply to them.

The survey results quantified specific concerns: 73.2% of learners cited the cost of data as a barrier and 69.5% cited internet issues; 75% of parents worried about the cost of data and 50% did not trust online learning for their children's education; 66.9% of school administrators felt an e-learning system would not be affordable to implement; and 66.7% of instructors felt an e-learning platform would increase their workload, with 80% citing poor network quality as a challenge.

Based on these findings, the group proposed concrete mitigations that were later folded into the system requirements: a low-data mode with offline access and compressed media to address cost concerns; low-bandwidth operation, recorded lessons, and caching to address network quality concerns; and secure login, certified instructors, and dedicated technical support to build trust in the platform.

4. Task 3: Requirements Analysis and Software Requirements Specification

Task 3 analyzed the data gathered in Task 2 in depth, reviewed and validated the requirements, identified issues, prioritized and classified the requirements, and produced a full Software Requirements Specification (SRS) for the Learn Bridge platform.

4.1 Analysis of Survey Findings

The survey data revealed a clear device and connectivity profile for the target users. Learners were found to mostly use smartphones, predominantly running Android, and to depend heavily on mobile data rather than fixed broadband. Power outages were reported as a frequent disruption to learning. Learners expressed a preference for multimedia learning content, but poor network quality and high data consumption were identified as major barriers. Learners also wanted more interaction with lecturers.

Parents indicated a strong desire for progress tracking of their children, while school administrators valued attendance and progress tracking and requested offline content support. Instructors reported experiencing network challenges themselves, and asked for recording capability, live chat features, online quizzes, and attendance tracking tools; most instructors reported using laptops for online teaching.

4.2 Review and Analysis of Requirements

The gathered requirements were reviewed for completeness, clarity, technical feasibility, and dependency relationships between requirements. This review surfaced a number of issues that needed to be resolved before the requirements could be finalized, including inconsistencies between what different stakeholder groups expected, ambiguous requirement statements that needed clarification, and gaps where information was missing entirely. Each of these issues was documented and addressed before the requirements were carried forward.

4.3 Prioritization of Requirements

Requirements were prioritized using a MoSCoW-style approach, weighing importance to users against technical feasibility within the project's time constraints. The table below summarizes the prioritization outcome.

Requirement	Importance	Feasibility	Priority
User authentication and role-based access (Learner, Instructor, Admin, Parent)	High	High	Must-have
Mobile-friendly and responsive interface	High	High	Must-have
Course/content upload and management	High	High	Must-have

Requirement	Importance	Feasibility	Priority
Recorded lessons (video/audio/text)	High	Medium	Must-have
Online quizzes and assessments	High	High	Must-have
Attendance and progress tracking	High	Medium	Must-have
Offline downloadable learning materials	High	Medium	Must-have
Low-bandwidth optimization (data-saving mode)	High	Medium	Must-have
Live video classes (basic implementation)	High	Low	Should-have
Live chat / Q&A during lessons	Medium	Medium	Should-have
Lecturer rating and feedback system	Medium	High	Should-have
Parent monitoring dashboard	Medium	Medium	Should-have
Advanced QoE-based adaptive streaming	High	Low	Nice-to-have
Real-time reconnection during live sessions	Medium	Low	Nice-to-have

Core functionality — authentication, content delivery, quizzes, and tracking — together with the offline and low-data features critical to low-resource settings, were treated as Must-have. Live video classes were retained but scoped down to a simplified implementation, while fully adaptive QoE-based streaming was deferred as a Nice-to-have given its technical complexity relative to the available time.

4.4 Classification of Requirements

Functional Requirements

- User registration, login, and role-based access for learners, instructors, parents, and administrators.
- Instructor content upload and management (video, audio, text, and slide-based materials).
- Recorded lessons accessible at any time, and online quizzes and assessments.
- Attendance tracking for both learners and instructors, and progress tracking based on activity and assessment data.
- Live classes with real-time chat and question-and-answer functionality.
- Learner ratings and feedback on instructors.
- Parent access to learner progress information, and administrator reports on attendance, performance, and engagement.

Non-Functional Requirements

- Mobile-friendly, responsive design optimised for Android, including low-end devices.
- Efficient operation under low-bandwidth and unstable network conditions, minimising data consumption.
- Offline access to learning materials through downloadable content.
- High reliability, availability, and fast response times (target: under 5 seconds).
- Usability for users with varying levels of digital literacy.
- Adaptive content delivery based on real-time network conditions.
- Data security enforced through authentication and strict role-based access control.
- Scalability to support many simultaneous users.

4.5 Software Requirements Specification (SRS) Summary

The SRS formalised the scope of the platform: a web-based adaptive e-learning system delivering content in multiple formats (video, audio, text), adjusting to network conditions, and providing offline capability for continuity in low-resource environments. Its objectives were to improve accessibility to education, enhance learner engagement, support instructors, and enable administrators and parents to monitor learning activity.

Constraints

- Network: the system must operate under low-bandwidth, unstable, and intermittent connectivity, including partial offline functionality.
- Device: optimisation for low-end Android smartphones as well as instructor laptops, with a lightweight, responsive interface.
- Performance: fast loading even on slow networks, with compressed and optimised multimedia content.
- Usability: a simple, intuitive, mobile-friendly interface usable by people with varying digital literacy.
- Content: multiple content formats supported, with automatic or manual compression before delivery.
- Technology: the system must be built as a Progressive Web App (PWA) using React for the frontend, Express.js on Node.js for the backend (to support real-time features such as live chat), and PostgreSQL for structured, relational data storage supporting role-based access control.

Key Assumptions

- Users have basic digital literacy and will not share accounts.
- Learners primarily use smartphones; instructors primarily use laptops; all devices support modern web browsers.

- Users have at least occasional, if unstable, internet access sufficient for periodic synchronization.
- Instructors will upload structured, well-formatted learning materials.
- Regular platform usage will allow the adaptive delivery mechanisms to function effectively.

This SRS, and the technology constraints it fixed (PWA + React + Node.js/Express + PostgreSQL), became the technical foundation for the system design work in Task 4 and the implementation work in Tasks 5 and 6.

5. Task 4: System Modelling and Design

Task 4 translated the functional and non-functional requirements from Task 3 into structural and behavioural architectural blueprints, using the Unified Modelling Language (UML) and structured systems analysis techniques. The design strictly followed the technology constraints established in the SRS: a Progressive Web App with a React frontend, a Node.js/Express backend, and a PostgreSQL database.

5.1 Use Case Diagram

Actors were classified by their level of interaction with the system: Learner and Instructor as primary, active initiators; Administrator as a secondary system maintainer; and Parent as a tertiary, passive observer who monitors progress without altering system data.

Twelve use cases were specified in detail, covering the full scope of the platform, including: logging in and credential verification; accessing adaptive content that adjusts to the learner's network bandwidth and device capability; taking quizzes; saving or caching content offline via the PWA service worker and IndexedDB for continued study during power outages or disconnections; viewing progress and attendance; participating in live, low-bandwidth text chat; uploading learning materials; tracking attendance (including offline module completions synced retroactively); managing courses; managing users and roles; and generating system-wide reports.

5.2 Class Diagram

The class diagram modelled the static structure of the backend and database. A User base entity centralises shared attributes (id, name, email, password hash, role) and core login/logout behaviour, specialised into Learner (with network-quality state and offline sync methods) and Instructor (with course-creation and attendance-tracking methods). A Course aggregates Modules through a composition relationship, and each Module exposes a method to fetch content adapted to the current QoE conditions. Assessments are composed within Modules.

To correctly represent many-to-many relationships in PostgreSQL, associative entities were introduced: a ProgressRecord linking Learner and Course (tracking attendance percentage and overall grade), and an AssessmentSubmission linking Learner and Assessment (tracking score and a sync status that distinguishes online submissions from those still pending in the offline cache).

5.3 Deployment Diagram

The deployment diagram mapped the software artefacts onto physical nodes. On the client side, the Learner's low-end smartphone runs the React PWA in a mobile browser together with a Service Worker (sync manager) and an IndexedDB local database for offline caching, while the Instructor's laptop runs the same PWA optimised for larger screens. On the server side, a cloud application server runs Node.js/Express, exposing both a REST API for standard data requests and a WebSocket server for low-latency live chat, backed by a dedicated PostgreSQL database server. Client-server traffic uses HTTPS and WSS (kept adaptive and fault-tolerant for the Learner path), while the application server communicates with the database over an internal TCP/IP connection.

5.4 Sequence Diagrams

Ten sequence diagrams were produced to model the system's dynamic behaviour, covering attendance tracking, authentication, course content access, instructor rating, in-lesson interaction, live sessions with adaptive streaming, network failure handling, progress tracking, recorded lesson downloads, and content upload.

The most critical of these flows — adaptive content access — works as follows: the learner requests a course module; the PWA service worker intercepts the request; if the device is offline, the service worker serves previously cached text and low-resolution materials directly from IndexedDB so learning continues uninterrupted; if the device is online, the request is forwarded to the backend together with the current network-quality signal, the backend queries PostgreSQL, evaluates network status, selects an appropriately adapted payload (e.g. stripping high-resolution video in favour of compressed audio or text), returns it to the service worker (which updates the offline cache for next time), and finally renders it to the learner.

5.5 Data Flow Diagram (Level 1)

The Level-1 DFD decomposed the platform into nine functional processes and four data stores. The data stores are the User Database, the Content Repository (video, audio, text, slides, recordings), the Assessment Store (quiz questions, answers, scores), and the Progress & Attendance store (session logs, quiz results, attendance, activity history).

The processes span user authentication and role management; content management and upload; adaptive content delivery (the platform's core differentiator, adapting resolution/compression/format to bandwidth); live session management; the assessment and quiz engine; attendance and progress tracking (aggregating live-session, assessment, and offline-sync data); and reporting for the administrator dashboard, each explicitly traced back to specific functional and non-functional requirements from the SRS.

5.6 Context Diagram

A system context diagram situated the platform within its external environment, identifying the external entities (Learners, Instructors, Parents, Administrators) and the data flows exchanged with the system, together with the key design decisions that followed from these boundaries.

Together, these six modelling artefacts gave the team a complete architectural blueprint, ready to be implemented in Task 5 (frontend) and Task 6 (database and backend).

6. Task 5: User Interface Design and Frontend Implementation

Task 5 covered the User Interface (UI) design and frontend implementation of the platform, now named Learn Bridge. The goal was to design and build an intuitive, responsive, and accessible interface for learners, instructors, parents, and administrators, compatible with mobile devices and low-bandwidth environments.

6.1 Brand Identity

Learn Bridge was defined as an adaptive e-learning platform for learners, instructors, parents, and administrators, with a brand identity built around five pillars: accessibility, simplicity, reliability, inclusiveness, and innovation. The logo combines education and technology concepts to represent knowledge-sharing, online learning, and connectivity between learners and instructors.

6.2 Visual Design

A consistent color system was defined to support both aesthetics and usability:

Colour	Purpose
Deep Dark Navy Blue	Establishes authority and a premium, enterprise feel.
Vibrant Teal / Cyan	Draws attention to high-priority items, interactive states, and healthy operational indicators.
Crisp Bright White	Maximizes contrast and readability for text and statistics while keeping a clean, professional look.
Soft Gray	Reduces visual noise by dimming static labels so changing data stands out.
Orange	Flags resource consumption or approaching limits without the alarm of a critical red error.

Modern, readable typography was chosen to remain legible across mobile phones, tablets, and desktop computers.

6.3 UI Components

The interface was built from a consistent set of reusable components: a navigation bar (Dashboard, Courses, Assessments, Attendance, Messages, Profile); a dashboard summarizing course progress, recent activity, upcoming lessons, and notifications; course cards showing title, instructor, progress, and a quick-access button; an assessment interface for viewing quizzes, submitting answers, and receiving scores; a progress tracking panel for learning progress, attendance, and performance statistics; a parent dashboard for monitoring a child's activity, reports, and attendance; and an administrator dashboard for user management, course management, report generation, and system monitoring.

All layouts were built responsively — using flexible grids, media queries, and adaptive layouts — to provide a consistent experience across smartphones, tablets, laptops, and desktop computers.

6.4 Frontend Technology Stack

Technology	Purpose
React 19	User interface library
Next.js 16	Application framework
TypeScript	Type safety
Tailwind CSS	Styling
Lucide React	Icon set
Zustand	State management

An authentication system was implemented using Zustand for state management, allowing users to log in and access role-specific dashboards and features.

6.5 Role-Based Dashboards

Four distinct, role-specific dashboards were implemented:

- Learner Dashboard — course progress, active courses, learning resources, live classes, and performance statistics.
- Instructor Dashboard — course management, student tracking, revenue statistics, and course analytics.
- Parent Dashboard — child monitoring, progress reports, activity tracking, and learning alerts.
- Administrator Dashboard — user management, platform statistics, revenue tracking, and system monitoring.

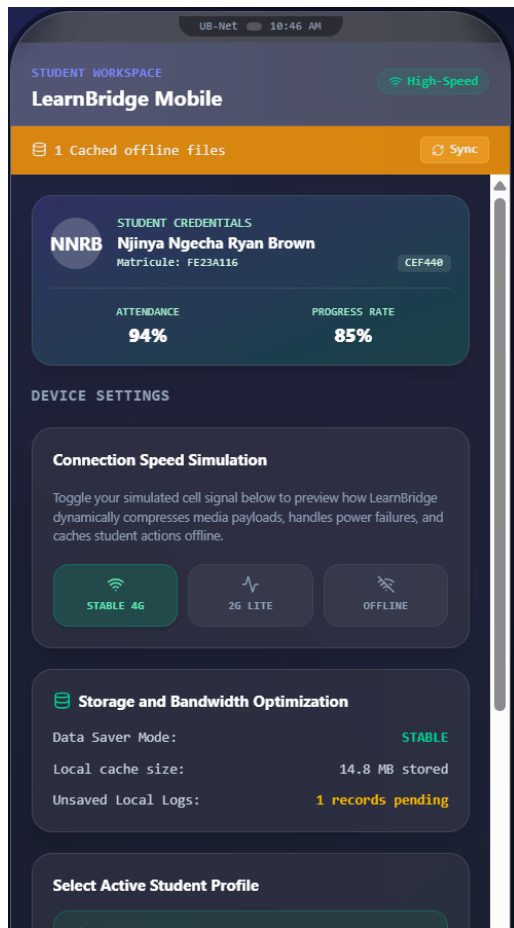


Figure 1: Learn Bridge Learner Portal, showing low-data mode, offline cache status, and quick access to courses, downloads, quizzes, and live classes.

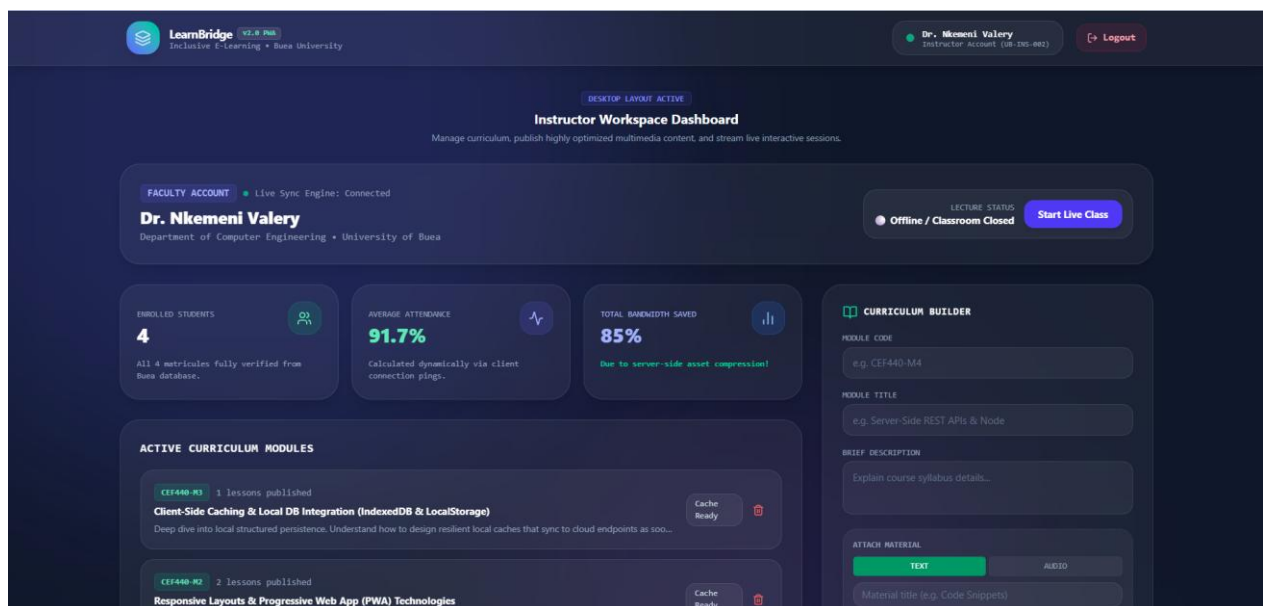


Figure 2: Learn Bridge Instructor Dashboard, showing course management and analytics.

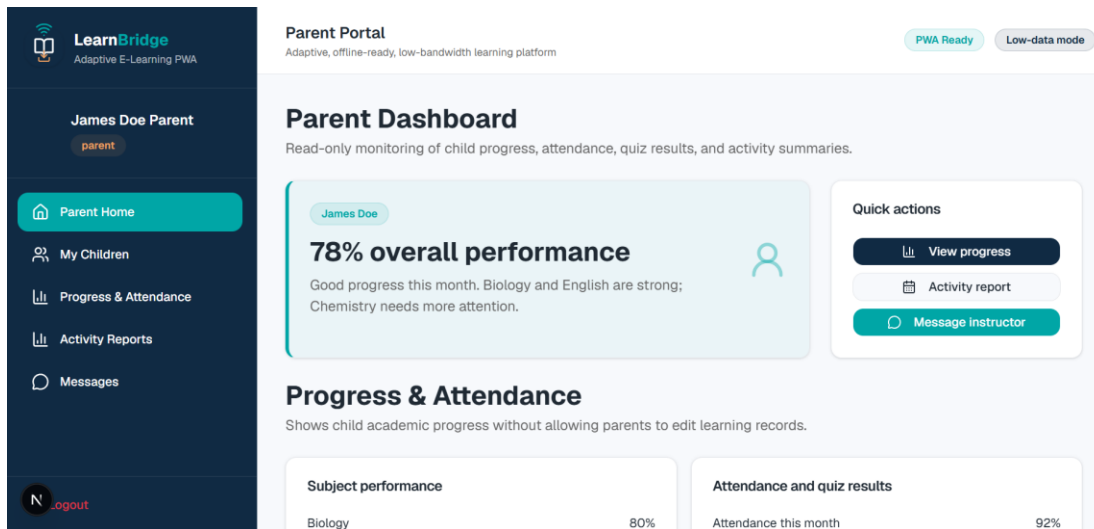


Figure 3: Learn Bridge Parent Dashboard, showing child monitoring and progress reports.

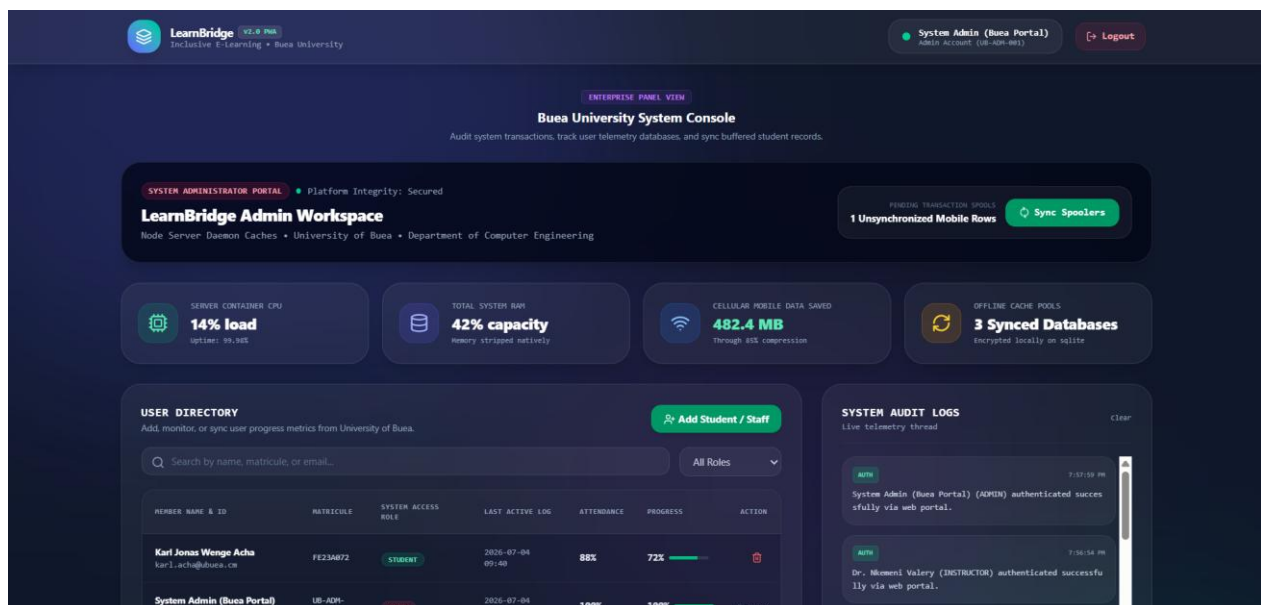


Figure 4: Learn Bridge Administrator Dashboard, showing platform-wide statistics and monitoring.

6.6 Outcome

By the end of Task 5, the Learn Bridge frontend had been implemented using React, Next.js, TypeScript, Tailwind CSS, and Zustand, with separate, purpose-built interfaces for each of the four user roles. The implementation followed responsive design principles throughout, providing an intuitive, scalable interface ready to be connected to a live backend and database in Task 6.

7. Task 6: Database Design and Backend Implementation

Task 6 completed the Learn Bridge stack by designing the relational database and implementing the backend API that connects the previously built frontend to persistent, real data. As the report for this

task notes, the frontend alone cannot permanently store application data: the frontend displays the interface, the backend receives and processes requests, and the database persists the data — the three layers work together to form a complete system.

The backend was built using PostgreSQL as the relational database, Prisma ORM as the data-access layer, Node.js and Express.js for the API, JWT for authentication, and bcrypt.js for password hashing, with the Next.js frontend consuming the API over HTTP.

7.1 Data Elements

Nine core data elements were identified from the system requirements and the four user roles (learner, instructor, parent, administrator):

- User — id, name, email, hashed password, role (LEARNER / INSTRUCTOR / PARENT / ADMIN), avatar, and timestamps. The role field determines which part of the system a user can access.
- Course — id, title, description, category, level, image, price, published status, and the owning instructor.
- Lesson — id, title, content, video URL, sequence order, duration, and parent course. The order field ensures course content is displayed in the correct sequence.
- Enrollment — links a learner to a course, forming the many-to-many relationship between learners and courses.
- Progress — tracks per-lesson completion for a learner within a course, enabling dashboard progress and completion percentages.
- Quiz — id, title, description, and the course it belongs to (a course can have many quizzes).
- Question — quiz question text, answer options (stored as JSON to allow a flexible number of choices), correct answer, and points.
- Quiz Result — learner, quiz, score, total points, submitted answers, and submission date, supporting performance tracking.
- Parent-Child — links a parent account to a learner (child) account, enabling parental monitoring of progress.

7.2 Conceptual Design

At the conceptual level, the system was organized into five logical layers: an Institutional layer (School, School Administrator, and Parent, providing oversight and accountability); a User layer (a generalised User specialised into Student and Instructor, supporting role-based access control); a Learning Process layer (Course, Enrollment, Learning Content, Offline Content, and Live Session, implementing multimedia support, offline access, and live interaction); an Assessment & Progress layer (Assessment, Assessment Result, Learner Progress, Instructor Feedback, and attendance tracking, supporting quizzes and parental monitoring dashboards); and a Quality & Reporting layer

(Network QoE and Report, ensuring adaptive delivery under poor network conditions and institutional reporting).

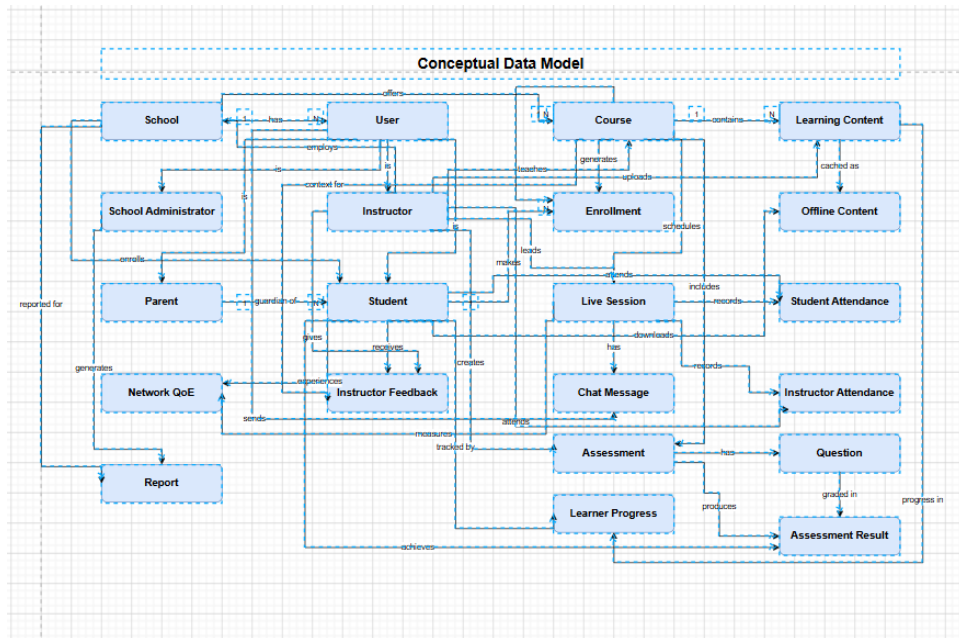


Figure 5: Conceptual data model of the Learn Bridge platform, organised into institutional, user, learning-process, assessment/progress, and quality/reporting layers.

7.3 Entity-Relationship Diagram

The logical ER diagram translated the conceptual layers into concrete database tables, attributes, keys, and relationships — including School and School Administrator; a generalized User table specialized into Student and Instructor; Course, Enrollment, Learning Content, Offline Content, and Live Session; Assessment, Question, Assessment Result, and Learner Progress; Student Attendance and Instructor Attendance; Instructor Feedback and Chat Message; and Network QoE and Report. Together these tables give the platform a fully normalized relational schema capable of supporting every functional requirement identified in the SRS.

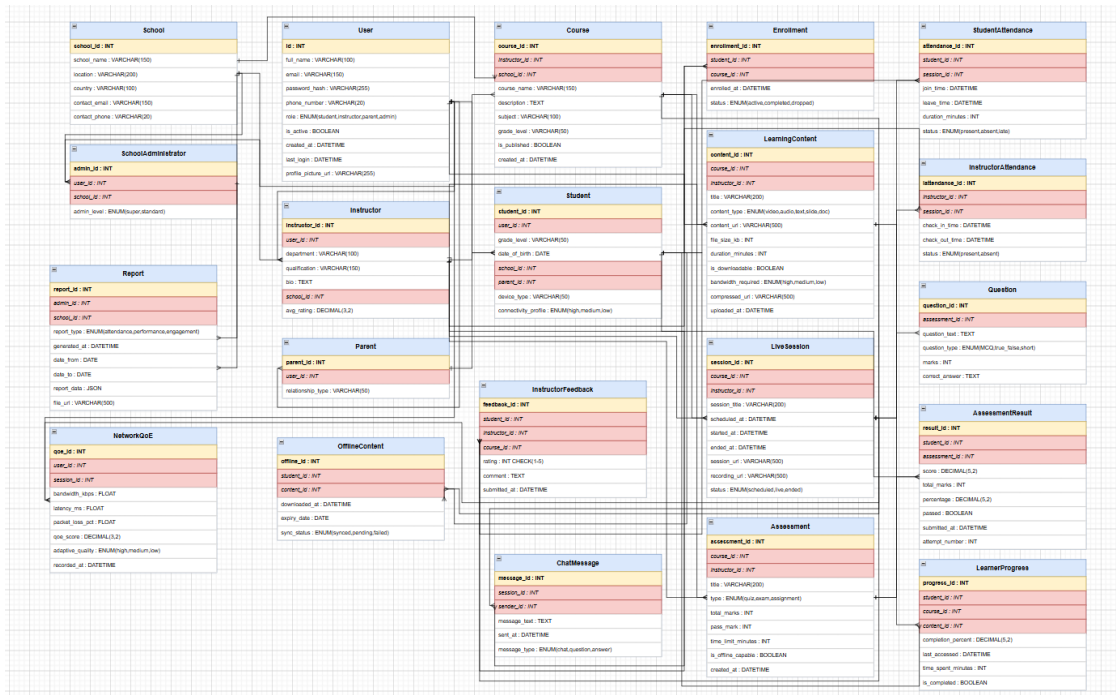


Figure 6: Entity-Relationship diagram implemented for the LearnBridge PostgreSQL database.

7.4 Database Implementation

The database was implemented in PostgreSQL, with the schema defined declaratively through Prisma (schema.prisma) and applied to the database via Prisma Migrate. Database constraints (primary keys, foreign keys, uniqueness, and enumerated types for fields such as role and status) were enforced at the schema level, and a seed script (Prisma/seed.js) was used to populate the database with representative sample data for development and testing.

7.5 Backend Implementation

The backend follows a layered Express.js structure, separating concerns into routes, controllers, middleware, and a shared Prisma client:

- Routes — auth Routes, course Routes, lesson Routes, quiz Routes, and dashboard Routes, each mapping HTTP endpoints to controller logic.
- Controllers — auth Controller, course Controller, lesson Controller, quiz Controller, and dashboard Controller, containing the business logic for each domain.
- Middleware — auth Middleware (verifying JWT tokens on protected routes) and role Middleware (enforcing role-based access control).
- lib/prisma.js — a shared, single instance of the Prisma client used across the application.

Authentication was implemented with JWT: users log in through the auth routes, receive a signed token, and must present it as an “Authorization: Bearerheader on subsequent requests to protected

endpoints. Passwords are hashed with `bcryptjs` before being stored, and are never stored or transmitted in plain text.

7.6 Connecting the Database to the Backend

Environment variables (managed through a `.env` file and `prisma.config.ts`) supply the database connection string used by the Prisma client. Example flows implemented and documented include fetching courses — where Prisma retrieves each course together with its instructor, lessons, and enrollment/lesson counts — and enrolling a learner, where an `upset` operation is used to guarantee a learner cannot be enrolled twice in the same course.

7.7 Testing

The backend was tested end-to-end using a running local server (`http://localhost:5000`). Tests covered a health-check endpoint (`GET /api/health`), the login endpoint (`POST /api/auth/login`), the courses endpoint (`GET /api/courses`), and the learner dashboard endpoint (`GET /api/dashboard/learner`), the latter requiring a valid Bearer token in the Authorization header. Each test confirmed the expected JSON response from the server.

7.8 Challenges Encountered

- PostgreSQL installation — the database could not initially be created because PostgreSQL was not installed; resolved by installing it and creating the learn bridge database.
- Prisma configuration — errors arose when Prisma could not locate the database URL or schema file; resolved by correctly configuring `schema.prisma`, `.env`, and `prisma config.ts`, and running Prisma commands from the correct backend directory.
- Authentication header — protected routes returned unauthorized errors when the “Bearer” prefix was omitted from the Authorization header; resolved by correctly storing and sending the JWT token from the frontend.

7.9 Outcome

By the end of Task 6, LearnBridge had a complete, working full-stack architecture: a relational PostgreSQL database modelling users, courses, lessons, enrollments, progress, quizzes, and results; a Node.js/Express backend exposing authenticated, role-aware REST endpoints through Prisma; and a React/Next.js frontend (from Task 5) ready to be wired to these live endpoints, replacing any static or mock data used during UI development.

8. Overall Conclusion and Recommendations

Across the six tasks documented in this report, Group 15 progressed from foundational research on mobile application development through to a working, end-to-end implementation of Learn Bridge, an adaptive e-learning platform designed specifically for Quality of Experience in low-resource settings.

The project followed a coherent and disciplined engineering path: Task 1 built the technical grounding needed to make informed architectural choices; Task 2 grounded the project in real stakeholder needs through surveys, observation, brainstorming, and competitor analysis; Task 3 turned that raw data into a validated, prioritized, and formally specified set of requirements; Task 4 translated those requirements into a complete UML-based system design; Task 5 delivered a polished, responsive, role-based user interface; and Task 6 delivered a functioning relational database and backend API, completing the full stack.

The recurring theme throughout every task was adaptivity for low-resource conditions: offline caching via service workers and Indexed DB, adaptive content quality based on network conditions, low-data modes, and lightweight, mobile-first design. This consistency between the requirements (Task 2/3), the design (Task 4), and the implementation (Task 5/6) demonstrates that the technical constraints identified early in the project were carried through faithfully to the finished system.

Link to the PWA: <https://learnbridge-191940838635.us-west1.run.app>

8.1 Current State of the Platform

- A fully designed relational database (PostgreSQL via Prisma) covering users, courses, lessons, enrollments, progress, quizzes, results, and parent-child relationships.
- A working Express.js/Node.js backend with authenticated, role-aware REST endpoints, tested for health, login, course listing, and dashboard retrieval.
- A responsive React/Next.js frontend with four distinct, role-based dashboards (Learner, Instructor, Parent, Administrator) styled to a consistent Learn Bridge brand identity.

